DOKUMEN DESAIN TEKNIS & ARSITEKTUR SISTEM OPS CONTROL (PT SUCOFINDO - TIC & OPERATIONAL)

Laporan Evaluasi Komprehensif, Pemetaan Monorepo, & Jawaban Evaluasi Penugasan

Penulis: Senior Fullstack Engineer | **Stack PoC:** React + Vite, Golang, SQLite, SSE, Docker

1. PROBLEM UNDERSTANDING & AKAR MASALAH BISNIS

Case Study 1 (Real-Time Order & Job Tracking): Akar masalah utama pada operasional *Testing, Inspection, and Certification* (TIC) PT Sucofindo adalah **fragmentasi informasi dan ketiadaan single source of truth**. Ketika status pekerjaan lapangan (inspeksi/sertifikasi) diperbarui teknisi, pembaruan tidak terdistribusi secara real-time ke Dispatcher, CS, dan Klien, memicu beban kerja CS berlebih dan keputusan operasional berbasis data kedaluwarsa (*stale state*).

Case Study 2 (Manajemen Produksi & Inventori Berbasis BOM): Akar masalah pada manajemen material adalah **inkonsistensi alokasi stok akibat decoupled inventory tracking**. Tanpa reservasi stok bahan baku yang terikat ke *Work Order* (WO) dan *Bill of Materials* (BOM), terjadi dua dampak fatal: *overselling/konflik stok* fisik gudang serta *inakurasi histori HPP* akibat revisi resep BOM master di kemudian hari.

Solusi Ops Control: Menyediakan visibilitas real-time berbasis event stream (Case 1) serta menjamin integritas transaksi alokasi material melalui *atomic stock reservation* dan *BOM snapshotting* (Case 2).

2. BUSINESS ASSUMPTIONS (ASUMSI DESAIN POC)

- **Aktor & Hak Akses:** Terdiri dari 4 peran (Klien, Dispatcher, Teknisi, Staf Gudang). Pada PoC, autentikasi disimulasikan melalui header request / skema routing interaktif.
- **Alur Status Order:** Transisi sah: `TODO → IN_PROGRESS → DONE`. Pembatalan (`CANCELLED`) hanya dari `TODO` atau `IN_PROGRESS`. Status `DONE` dan `CANCELLED` bersifat terminal (terkunci).
- **Work Order & BOM:** Satu WO memproduksi satu Produk Jadi berbasis BOM *single-level* (tanpa sub-assembly rekursif). Kuantitas material wajib positif.
- **Satuan Material:** Disimpan dalam *base unit* dasar (`kg`, `pcs`, `liter`). Konversi satuan terjadi di tingkat aplikasi.

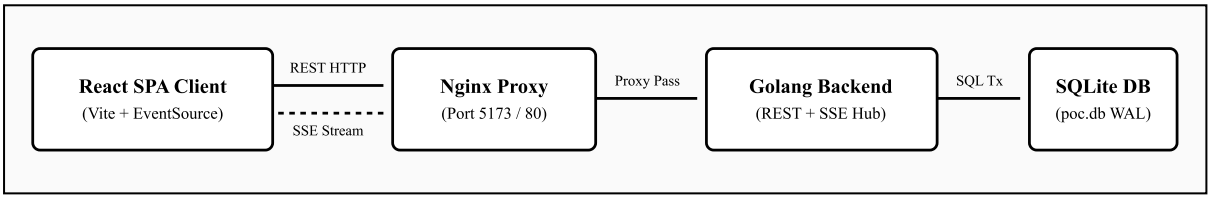
3. SCOPE: IN / OUT

Kategori	Fitur / Komponen	Alasan Teknis & Bisnis
In Scope (Terimplementasi)	Order Tracking (Create, Update Status, List), SSE stream broadcast, Material Stock View, Work Order Creation & Atomic Completion, Docker Compose environment.	Membuktikan pembaruan real-time dan alokasi stok yang konsisten dalam PoC.

Kategori	Fitur / Komponen	Alasan Teknis & Bisnis
In Scope (Rancangan Arsitektur)	Replay SSE via `Last-Event-ID`, Optimistic Concurrency Control, Redis Pub/Sub multi-pod broadcast, BOM Snapshotting & Cancel WO.	Didesain secara arsitektural untuk menjawab evaluasi skala produksi.
Out of Scope	User Authentication (OAuth2/JWT), Dynamic Multi-level BOM Tree, Partial WO Completion & Scrap, Automated CI/CD Pipeline.	Dieliminasi untuk menjaga fokus pada integritas data operasional PoC.

4. SYSTEM ARCHITECTURE & DIAGRAM KOMUNIKASI

Arsitektur Ops Control menggunakan pola *Decoupled Monolith* untuk efisiensi deployment dan kemudahan skabilitas:



- **Frontend:** React SPA berkomunikasi via REST HTTP untuk mutasi data dan menerima stream real-time melalui *EventSource* (SSE).
- **Backend:** Golang HTTP Server (Native) mengelola logika bisnis, transaksi DB, dan in-memory SSE Broadcast Hub.
- **Database & Infra:** SQLite dalam *WAL Mode* untuk efisiensi read/write tinggi, dibungkus dalam Docker Compose.

5. DATABASE DESIGN & ENTITY RELATIONSHIP (ERD)

Struktur relasi entitas database PoC disusun sebagai berikut:

- `orders (1) —< (N) order_events` : Satu order memiliki banyak log audit event transisi status.
- `products (1) —< (N) bom_items (N) >— (1) materials` : Struktur Bill of Materials memetakan kebutuhan bahan baku untuk produk jadi.
- `products (1) —< (N) work_orders` : Satu master produk dapat diproduksi melalui banyak Work Order.

Entitas utama mencakup `orders`, `order_events` (audit trail mencakup `actor_role` & `ip_address`), `materials` (stok fisik `on_hand` & alokasi `reserved`), `products`, `bom_items`, dan `work_orders`.

6. PROCESS FLOW

Alur Order Tracking (Case 1): Request `POST /api/orders/status` membawa `expected_version` & `idempotency_key`. Backend membuka transaksi DB → memverifikasi versi → mengupdate status order & menaikkan versi → mencatat event audit ke `order_events` → commit transaksi → broadcast payload ke SSE Hub → UI terupdate tanpa reload.

Alur Production & Work Order (Case 2): Pembuatan WO memeriksa stok `Available` seluruh komponen BOM. Jika mencukupi, sistem melakukan **Reservasi Material** (`reserved = reserved + req_qty`) secara atomik. Saat WO `DONE`, transaksi mengurangi `on_hand` & `reserved` bahan baku, serta menambah `on_hand` Barang Jadi.

7. JAWABAN PERTANYAAN DESAIN WAJIB — CASE STUDY 1

1. **SSE vs WebSocket vs Polling:** SSE dipilih karena alur tracking bersifat *unidirectional* (server ke klien). SSE lebih sederhana karena berjalan native di atas HTTP/1.1 biasa (tanpa overhead protocol upgrade WebSocket), bebas masalah proxy/firewall, dan memiliki protokol bawaan *auto-reconnect*. Polling ditolak karena pemborosan overhead HTTP.
2. **Mekanisme Reconnect & Missed Events:** Saat koneksi terputus dan terhubung kembali, browser otomatis mengirimkan header `Last-Event-ID` (atau query param `?last_event_id=...`). Server membaca ID tersebut dan memutar ulang (*replay*) seluruh event di `order_events WHERE id > Last-Event-ID`.
3. **Idempotency & Event Ordering:** Request status menyertakan `Idempotency-Key` yang diperiksa keunikannya di DB. Request duplikat akan mengembalikan respon cache tanpa mengeksekusi ulang efek samping. Urutan event dijamin oleh urutan *auto-increment primary key* transaksi database.
4. **Concurrency Control:** Menerapkan *Optimistic Concurrency Control* menggunakan kolom `version` (`UPDATE orders SET status=?, version=version+1 WHERE id=? AND version=?`). Jika baris terupdate = 0, backend menolak dengan HTTP 409 Conflict.
5. **Scaling 3 Pod Backend:** Di lingkungan multi-instance, event di-publish ke **Redis Pub/Sub** channel. Setiap pod backend bertindak sebagai subscriber yang memancarkan event ke koneksi SSE lokal masing-masing.
6. **Audit Trail:** Tabel `order_events` bersifat *append-only* mencatat `who (actor_role)`, `when`, `from_status`, `to_status`, `idempotency_key`, dan `ip_address`.

8. JAWABAN PERTANYAAN DESAIN WAJIB — CASE STUDY 2

1. **On Hand vs Reserved vs Available:**
   • **On Hand:** Stok fisik riil gudang. | **Reserved:** Stok yang dialokasikan untuk WO aktif.
   • **Available:** Stok bersih yang aman untuk alokasi baru (`Available = On Hand - Reserved`).
2. **Pencegahan Race Condition:** Pengecekan `Available >= Required` dilakukan di dalam transaksi database berkunci (*Immediate Write Transaction* pada SQLite / *SELECT FOR UPDATE* pada PostgreSQL).

3. **Atomic Database Transaction:** Pelepasan reservasi material dan penambahan barang jadi dibungkus dalam transaksi `BEGIN ... COMMIT`. Kegagalan pada salah satu bahan akan memicu `ROLLBACK` menyeluruh.
4. **BOM Versioning & Snapshot:** Komposisi material disalin ke kolom `bom_snapshot` (JSON) saat WO diterbitkan, sehingga revisi BOM master di kemudian hari tidak mengubah transaksi WO yang sedang berjalan.
5. **Handling Partial Completion, Scrap, & Stock Opname (Planned):**
   • *Partial Completion:* Mengurangi reservasi proporsional dan menambah barang jadi bertahap.
   • *Scrap:* Mencatat bahan rusak ke `material_adjustments` yang mengurangi `on_hand` tanpa menambah barang jadi.
   • *Stock Opname:* Penyesuaian fisik mengupdate `on_hand` dan mencatat selisih ke tabel audit variance.

9. TABEL ANALISIS EDGE CASES & MITIGASI KOMPREHENSIF

Skenario Edge Case	Risiko Bisnis	Penanganan Sistem (Technical Mitigation)	Status Implementasi
Request status terkirim 2x (network retry)	Duplikasi log audit status order	Pengecekan `Idempotency-Key` pada DB unique constraint. Request kedua mengembalikan respon cache.	Rancangan Arsitektur
Dua admin mengubah order bersamaan	Perubahan status menimpa (Lost Update)	Verifikasi `expected_version`. Request kalah mengalami rejection HTTP 409 Conflict.	Rancangan Arsitektur
Koneksi SSE terputus saat streaming	Klien kehilangan update status terbaru	Pengiriman `Last-Event-ID` via header/query param saat reconnect untuk replay event historis.	Rancangan Arsitektur
Bahan baku kurang saat Work Order dibuat	Gagal produksi & partial allocation tertahan	Pengecekan ketersediaan `Available` seluruh BOM dalam transaksi atomik. Jika ada 1 bahan kurang, transaksi abort.	Terimplementasi di PoC
Revisi BOM Master saat Work Order aktif	Inakurasi kalkulasi kebutuhan material WO	Pembekuan komposisi material ke dalam `bom_snapshot` JSON saat WO diterbitkan.	Rancangan Arsitektur
Pembatalan Work Order di tengah alokasi	Stok Reserved terkunci selamanya	Alur cancel WO melepaskan stok teralokasi: `reserved = reserved - req_qty`.	Rancangan Arsitektur

10. TRADE-OFFS & INFRASTRUCTURE READINESS

- **SQLite vs PostgreSQL:** SQLite dipilih untuk PoC tanpa overhead instalasi DB server. Untuk lingkungan produksi multi-user skala besar, migrasi ke PostgreSQL mutlak dilakukan untuk mendukung *high-concurrency write*.
- **Container & Cloud Delivery:** Environment `docker-compose.yml` menggabungkan service frontend React (Nginx) dan backend Golang. Manifest Kubernetes (Deployment, Service, ConfigMap, Secret, & Liveness Probe `/health`) siap digunakan untuk deployment cloud.

11. WHAT'S NEXT (RENCANA PENGEMBANGAN 2 MINGGU)

1. **Minggu 1:** Migrasi persistence ke PostgreSQL, integrasi Redis Pub/Sub untuk SSE scaling multi-pod, dan penambahan autentikasi JWT + RBAC.
2. **Minggu 2:** Implementasi fitur produksi Partial Work Order Completion, pencatatan Material Scrap, dan Load Testing untuk SSE stream.